МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ

Тооцоолол ба өгөгдлийн ухаан

#2 - Python программчлал

(string, search, function, scope)

Өнгөрсөн хичээлээр

- Тооцоолол гэж юу вэ
- Python хэлний суурь ойлголт
- Математик операторууд
- Хувьсагч, үндсэн өгөгдлийн төрлүүд
- Control Flow - нөхцөл шалгах, давталт

Өнөөдрийн хичээл

- Тэмдэгт мөртэй ажиллах
- Тааж, шалгах алгоритм
- Ойролцоолох
- Хэсэглэн хайх арга
- Программын бүтэц
- Функц, тодорхойлолт
- Үйлчлэх хүрээ

Тэмдэгт мөр (String)

- Тэмдэгтийн **дараалал**
- ==, >, < зэрэг оператороор харьцуулалт хийж болно
- len() функцээр тэмдэгт мөрийн **урт**ыг мэдэж болно

```
s = "abc"
len(s) # => 3 гэж буцаана
```

Тэмдэгт мөр (String)

- Дөрвөлжин хаалт нь заасан байрлал дахь утгыг авахын тулд **индексээр хандана** (**indexing**)

```
s = "abc"
index:        0  1  2     ←   үргэжл 0-ээс эхэлнэ
index:       -3 -2 -1     ←   сүүл үргэлж -1-ээс эхэлнэ

s[0]    →   утга "a"
s[1]    →   утга "b"
s[2]    →   утга "c"
s[3]    →   индекс хэтэрсэн тул алдаа өгнө
s[-1]   →   утга "c"
s[-2]   →   утга "b"
s[-3]   →   утга "a"
```

Тэмдэгт мөр (String)

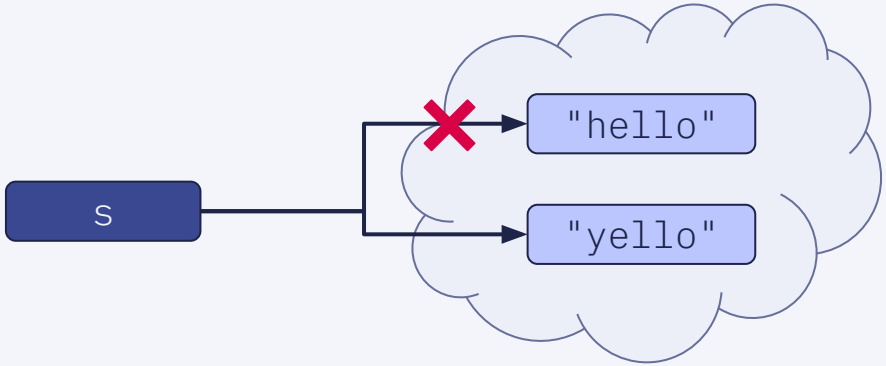
- Тэмдэгт мөрийг **зүсч** (**slice**) хийж болно, `[start:stop:step]`
- Хоёр тоо өгвөл, `[start:stop], step=1`
- Тоо бичихгүй дан тодорхойлох хоёр цэгээр байсан ч болно

```
s = "abcdefgh"
s[3:6]    →  "def" буцаана, s[3:6:1] адил
s[3:6:2]     →  "df" буцаана
s[::]        →  "abcdefgh" буцаана, s[0:len(s):1] адил
s[::-1]      →  "hgfedcba" буцаана, s[-1:-(len(s)+1):-1]
s[4:1:-2]→  "ec" буцаана
```

Тэмдэгт мөр (String)

Тэмдэгт мөр **хувирахгүй** (**immutable**) - утгыг нь өөрчилж болохгүй

```
s = "hello"
s[0] = 'y'                  →  өөрчилж болохгүй, алдаа өгнө
s = 'y' + s[1:len(s)]       →  s шинэ string объект заана
```



Тэмдэгт мөр болон давталт

- Дараах хоёр кодын хэсэг яг ижил үйлдэл хийнэ
- Доорх нь илүү “pythonic”

```
s = "abcdefgh"
for index in range(len(s)):
    if s[index] == 'i' or s[index] == 'u':
        print("There is an i or u")

for char in s:
    if char == 'i' or char == 'u':
        print("There is an i or u")
```

Жишээ

```
an_letters = "aefhilmnorsxAEFHILMNORSX"
word = input("I will cheer for you! Enter a word: ")
times = int(input("Enthusiasm level (1-10): "))
i = 0
while i < len(word):
    char = word[i]
    if char in an_letters:
        print("Give me an " + char + "! " + char)
    else:
        print("Give me a " + char + "! " + char)
    i += 1
print("What does that spell?")
for i in range(times):
    print(word, "!!!")
```

```
for char in word:
```

Тааж, шалгах (guess-and-check)

- Доорх процессыг **exhaustive enumeration** гэнэ

- Бодлого өгөгдөхөд:
  - Хариуг **таах боломжтой**
  - Хариуг **зөв эсэхийг шаглах боломжтой**
  - Шийдлийг олох эсвэл бүх утгыг тааж дуусах хүртэл таана

Тааж, шалгах - Жишээ

```
# куб язгуур олох
cube = 8
for guess in range(cube + 1):
    if guess**3 == cube:
        print(cube, "-н куб язгуур бол", guess)
```

Тааж, шалгах - Жишээ

```
cube = 8
guess = 0
for guess in range(abs(cube) + 1):
    if guess**3 >= abs(cube):
        break
if guess**3 != abs(cube):
    print(cube, "бол бүхэл тооны куб биш")
else:
    if cube < 0:
        guess = -guess
print(str(cube) + "-н куб язгуур бол " + str(guess))
```

Шийдлийг ойролцоолох (Approximation)

- **Хангалттай сайн шийд** олох
- Ямар нэг утгаас эхлэн тааж, маш **бага утгаар** нэмэгдүүлнэ
- **Бага `epsilon`** утгын хувьд `|guess³ - cube| >= epsilon` нөхцөл биелэхгүй болох хүртэл давтана

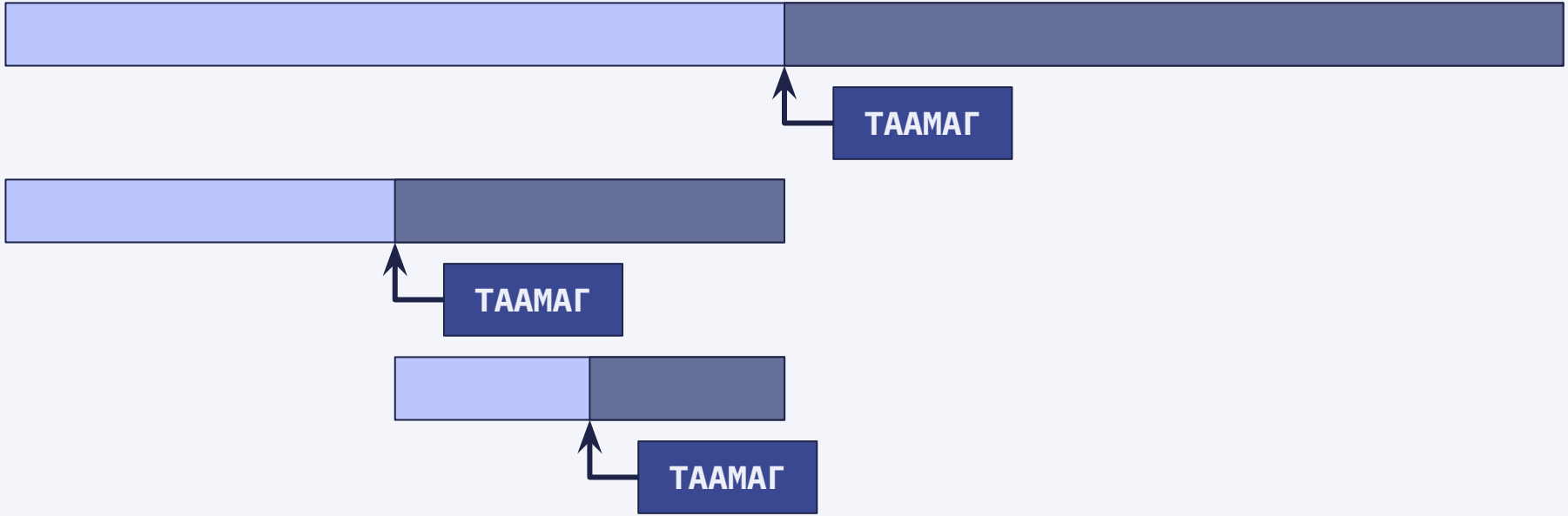
- Нэмэгдүүлэх утгыг багасгах → Удаан программ
- Epsilon-г ихэсгэх → Нарийвчлал бага

Шийдлийг ойролцоолох

```
cube = 27
epsilon = 0.01
guess = 0.0
increment = 0.0001
num_guesses = 0

while abs(guess**3 - cube) >= epsilon:
    guess += increment
    num_guesses += 1
    print('num_guesses =', num_guesses)

if abs(guess**3 - cube) >= epsilon:
    print('Failed on cube root of', cube)
else:
    print(guess, 'is close to the cube root of', cube)
```

Хэсэглэн хайх (хоёртын (Bisection search))

- Давталт бүр хайх хүрээг хагасаар багасгана
- Шинэ таамгийг тэг дундаас нь сонгоно



Хэсэглэх хайх жишээ - Куб язгуур

```
cube = 27
epsilon = 0.01
num_guesses = 0
low = 0
high = cube
guess = (high + low) / 2.0
while abs(guess**3 - cube) >= epsilon:
    if guess**3 < cube:
        low = guess
    else:
        high = guess
    guess = (high + low) / 2.0
    num_guesses += 1
print("num_guesses =", num_guesses)
print(guess, "is close to the cube root of", cube)
```

Хэсэглэн хайх

- Хайлтын муж
  - Эхний таамаг: $N/2$
  - Хоёр дахь таамаг: $N/4$
  - k дугаар таамаг: $N/2^k$
- $\log_2 N$ алхмын дараа хариутай таарна
- Хэсэглэж хайх нь оролтын утгаас хамаарч тогтмол хэмжээгээр өөрчлөгддөг функцэд л ажиллана

Программын бүтэц - Задлах, хийсвэрлэх

Кодоо хэрхэн бичиж байна?

- Бид одоогийн байдлаар
  - Нэг файлд
  - Дараалласан зааварчилгаа
  - Өөр тооцоолол хийх болгонд шинэ файлд бичнэ
- Энэ нь
  - Жижиг асуудалд, хялбар шийдэл
  - Том асуудалд, замбараагүй
  - Үйлдлүүдийг ялгаж уншихад төвөгтэй

Сайн программ бичих эхлэл

- Их код бичих нь ихэнх тохилдолд сайн зүйл биш
- Өргөн хэрэглэгдэж буй үйлдлээ **задлах** (**decomposition**), **хийсвэрлэх** (**abstraction**) шаардлагатай болдог
- Үүнийг **функц** (**function**) бичих замаар шийдэх боломжтой

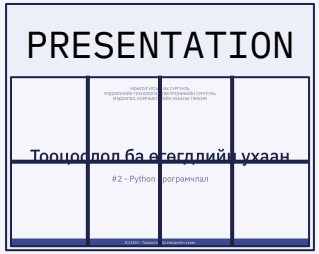
Жишээ - Проектор

- Проектор бол хар хайрцаг
- Дотор нь юу болоод байгаа юм бүү мэд
- Гэхдээ бид хэрхэн харьцахыг мэднэ: оролт/гаралт

ABSTRACTION IDEA: Бид проекторыг хэрэглэхийн тулд хэрхэн ажилладагийг мэдэх шаардлагагүй

Жишээ - Проектор

- Маш том зураг харуулахын тулд зургийг хувааж олон проекторт жижиг ажил болгож өгдөг
- Проектор болгон өөр өөр гаралт гаргана
- Бүх проектор нийлж, нэг том зураг харуулна



DECOMPOSITION IDEA: Өөр өөр төхөөрөмж нийлж ажиллан нэгдсэн зорилгод хүрэх

Бүтэцлэх - Задлах (Decomposition)

- Программчлалд кодыг **модуль** (**modules**) болгон задалдаг
  - Бие даасан (**self-contained**)
  - Код салгах (**break up**) зорилготой
  - Дахин ашиглахад (**reusable**) зориулсан
  - Кодыг эмх цэгцтэй (**organized**) байлгана
  - Кодыг уялдаа холбоотой (**coherent**) байлгана
- Энэ лекцээр кодыг задлахдаа **функц** (**function**) гэх ойлголтод тулгуурлана
- Хэдэн лекцийн дараа код задлах өөр нэг арга болох **класс** (**class**) хэмээх ойлголтыг авч үзнэ

Агуулга далдлах - Хийсвэрлэл (Abstraction)

- Өмнө үзүүлсэн жишээнд проектор хэрэглэхийн тулд хэрхэн проектор хийх талаар бид мэдэх шаардлагагүйг ойлгосон
- Программчлалд үүнтэй төстэй кодын хэсгийг **хар хайрцаг** (**black box**) гэж бодож болно
  - Агуулгыг харах боломжгүй
  - Агуулгыг харах шаардлагагүй
  - Агуулгыг харахыг хүсэхгүй
  - Нарийн төвөгтэй агуулгыг нуух
- Хийсвэрлэлийг **функцийн тодорхойлолт** (**function specifications**) эсвэл **docstrings**-р гүйцэтгэнэ

Функц (Function)

- Дахин ашиглах кодын хэсгийг функц хэмээн нэрлэнэ
- Функцийг дуудахаас (“**called**” эсвэл “**invoked**”) нааш программд ажиллахгүй
- Функцийн шинж:
  - Нэртэй
  - Параметртай (**parameters**) (0 эсвэл түүнээс олон)
  - Заавар, тайлбартай (**docstrings**) (заавал биш, гэхдээ бичвэл зүгээр)
  - Их биетэй (хэрэгжүүлэлт буюу агуулга)
  - Ямар нэг утга буцаадаг (**return**)

Функц хэрхэн бичих, дуудах вэ

keyword нэр параметр

```
def is_even(i):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print("inside is_even")
    return i % 2 == 0
```

заавар, тайлбар

их бие

```
is_even(3)
```

кодын өөр нэг хэсэгт функцийг нэрээр нь шаардлагатай аргументийн хамт дуудаж, ажиллуулна

Функцийн их бие

```
def is_even(i):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print("inside is_even")
    return i % 2 == 0
```

тушаал ажиллана

keyword

илэрхийллийг тооцоолж,
утга буцаана

Хувьсагчийн үйлчлэх хүрээ (Variable scope)

- Хуурмаг параметр нь функц дуудагдах үед жинхэнэ параметр болж, дуудсан үеийн утгыг авна
- Функц ажиллаж эхлэхэд шинэ хүрээ, орчин (**scope/frame/environment**) үүснэ

хуурмаг параметр

```
def f(x):
    x = x + 1
    print("in f(x): x =", x)
    return x
```

функцийн тодорхойлолт

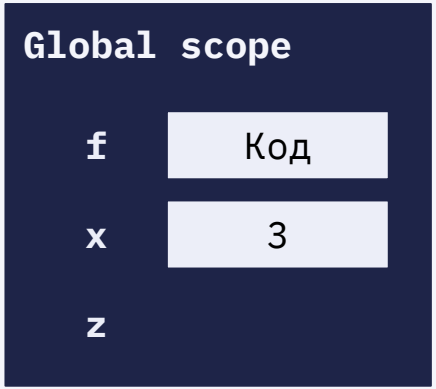
```
x = 3
z = f(x)
```

жинхэнэ параметр (аргумент)

үндсэн программ
* x хувьсагч зарлана
* f(x) функцийг дуудна
* функцийн буцаасан утгыг z хувьсагчид олгоно

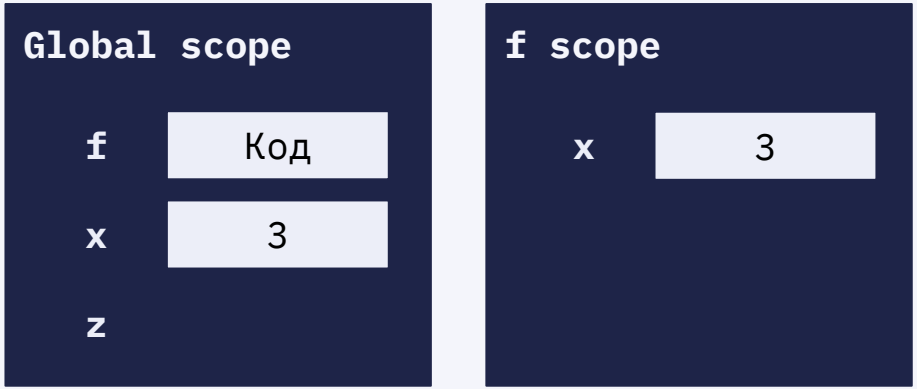
Хувьсагчийн үйлчлэх хүрээ (Variable scope)

```
def f(x):
    x = x + 1
    print("in f(x): x =", x)
    return x

x = 3
z = f(x)
```



Хувьсагчийн үйлчлэх хүрээ (Variable scope)

```
def f(x):
    x = x + 1
    print("in f(x): x =", x)
    return x

x = 3
z = f(x)
```



Хувьсагчийн үйлчлэх хүрээ (Variable scope)

```
def f(x):
    x = x + 1
    print("in f(x): x =", x)
    return x
```

```
x = 3
z = f(x)
```

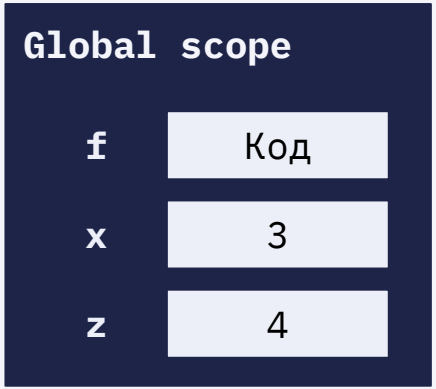
Хувьсагчийн үйлчлэх хүрээ (Variable scope)

```
def f(x):
    x = x + 1
    print("in f(x): x =", x)
    return x
```

```
x = 3
z = f(x)
```

Хувьсагчийн үйлчлэх хүрээ (Variable scope)

```
def f(x):
    x = x + 1
    print("in f(x): x =", x)
    return x

x = 3
z = f(x)
```



`return` байхгүй үед юу болох вэ?

```
def is_even(i):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print("inside is_even")
    i % 2 == 0
```

return statement байхгүй

- return байхгүй функц дууссан тохиолдолд **Python хэл None утга буцаана**
- None гэдэг нь утга байхгүйг илэрхийлдэг утга

`return` vs `print`

- Зөвхөн функц **дотор** ашиглана
- Функц зөвхөн **нэг л удаа** return-г ажиллуулна
- return-ий ард бичигдсэн код хэзээ ч ажиллахгүй
- Утга өгөгдсөн бол түүнийг **функц дуудагч руу буцаана**

- Функцийн **гадна, дотно** ашиглаж болно
- Нэг функц **хэдэн ч** print дуудаж болно
- print дуудагдсан ч түүний арын код ажиллана
- Утга өгөгдсөн бол **консоль руу хэвлэнэ**

Функцийг аргументаар дамжуулах

- Функцийн аргумент юу ч байж болно, функц ч зөвшөөрнө

```
def func_a():
    print("inside func_a")

def func_b(y):
    print("inside func_b")
    return y

def func_c(z):
    print("inside func_c")
    return z()

print(func_a())

print(5 + func_b(2))

print(func_c(func_a))
```

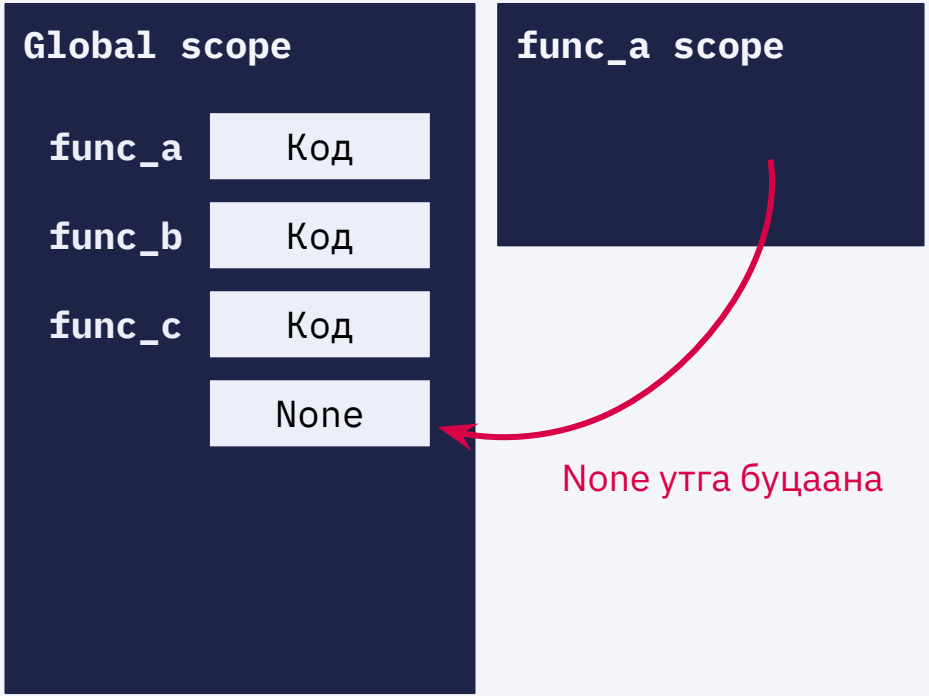
func_a параметр авахгүй

func_b нэг параметр авна

func_c нэг параметр авна, өөр функц дамжуулсан

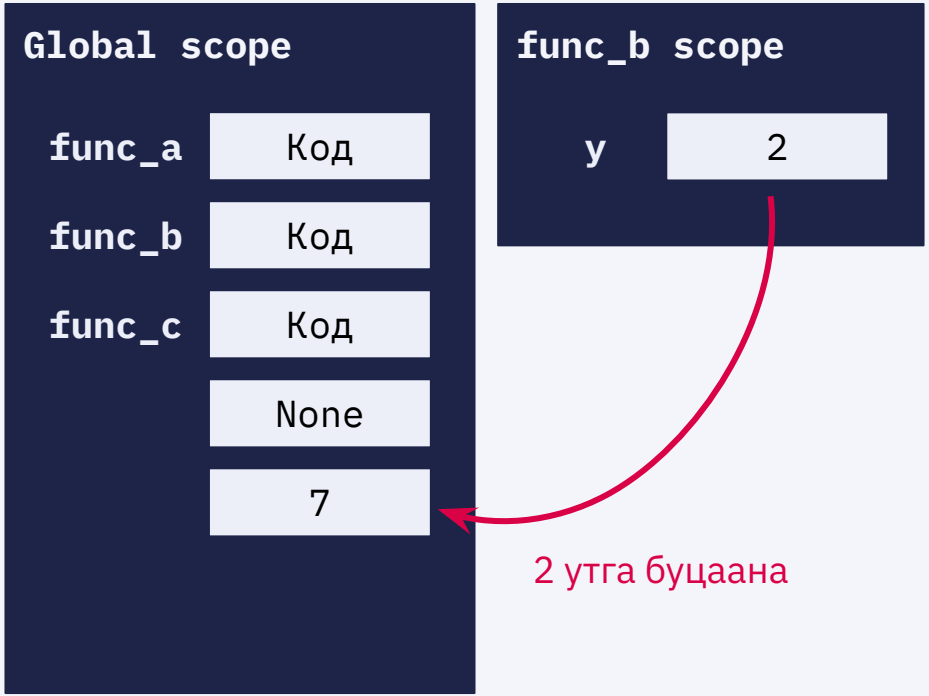
Функцийг аргументаар дамжуулах

```
def func_a():
    print("inside func_a")

def func_b(y):
    print("inside func_b")
    return y

def func_c(z):
    print("inside func_c")
    return z()

print(func_a())

print(5 + func_b(2))

print(func_c(func_a))
```



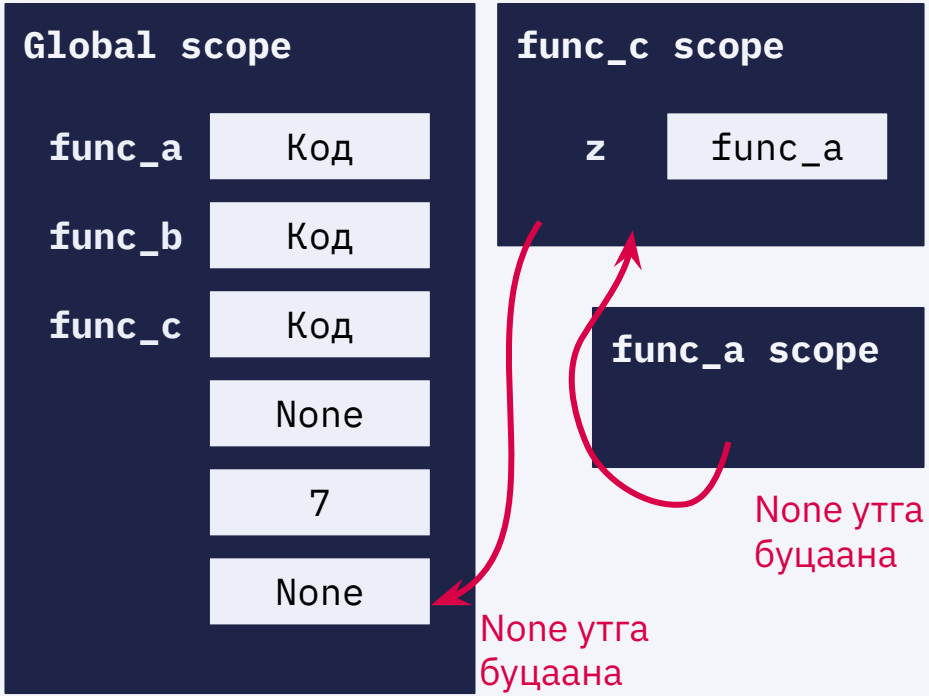
Функцийг аргументаар дамжуулах

```
def func_a():
    print("inside func_a")

def func_b(y):
    print("inside func_b")
    return y

def func_c(z):
    print("inside func_c")
    return z()

print(func_a())

print(5 + func_b(2))

print(func_c(func_a))
```



Функцийг аргументаар дамжуулах

```
def func_a():
    print("inside func_a")

def func_b(y):
    print("inside func_b")
    return y

def func_c(z):
    print("inside func_c")
    return z()

print(func_a())

print(5 + func_b(2))

print(func_c(func_a))
```



Үйлчлэх хүрээ - Жишээ

- Функцийн дотроос гадна зарласан хувьсагчид **хандаж чадна**
- Функцийн дотроос гадна зарласан хувьсагчийг **өөрчилж чадахгүй**. global хувьсагч ашиглаж болох боловч муу арга

```
def f(y):
    x = 1
    x += 1
    print(x)

x = 5
f(x)
print(x)
```

x-г f-н хүрээнд дахин тодорхойлсон.

өөр x объект

```
def g(y):
    print(x)
    print(x + 1)

x = 5
g(x)
print(x)
```

гадна зарласан x

g доторх x нь g-г дуудсан газрынх

```
def h(y):
    x += 1

x = 5
h(x)
print(x)
```

UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

Үйлчлэх хүрээ - Жишээ

- Функцийн дотроос гадна зарласан хувьсагчид **хандаж чадна**
- Функцийн дотроос гадна зарласан хувьсагчийг **өөрчилж чадахгүй**. global хувьсагч ашиглаж болох боловч муу арга

```
def f(y):
    x = 1
    x += 1
    print(x)

x = 5
f(x)
print(x)
```

```
def g(y):
    print(x)
    print(x + 1)

x = 5
g(x)
print(x)
```

```
def h(y):
    x += 1

x = 5
h(x)
print(x)
```

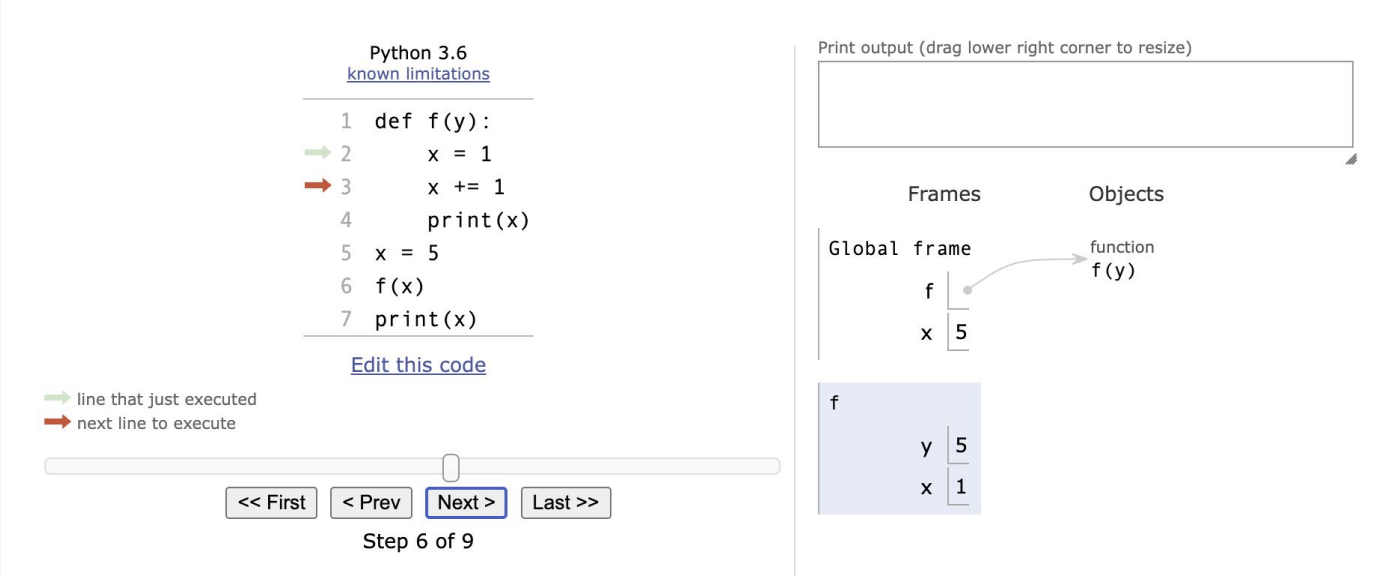
global/main program scope

Өөрсдөө туршиж үзэх

Python Tutor-н тусламжтай кодоо алхам алхмаар нь ажиллуулж, ямар объект хаана үүсч байгааг харах боломжтой.

https://pythontutor.com/

Өөрсдөө туршиж үзэх

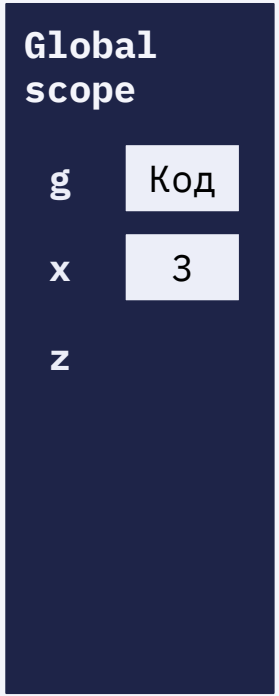


https://pythontutor.com/

Үйлчлэх хүрээ - Жишээ

```
def g(x):
    def h():
        x = "abc"

    x = x + 1
    print("g: x =", x)
    h()
    return x

x = 3
z = g(x)
```

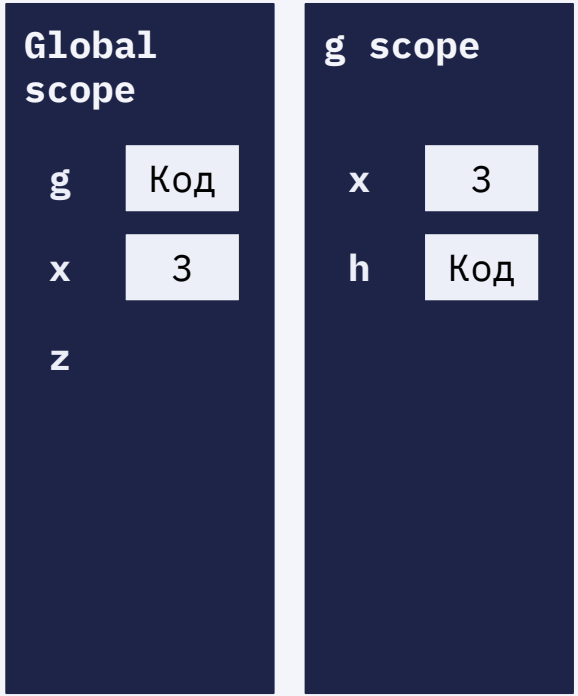
g код



Үйлчлэх хүрээ - Жишээ

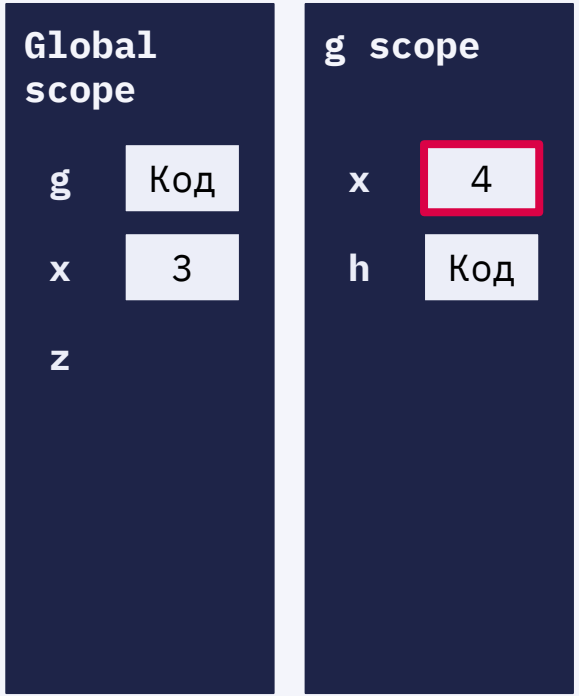
```
def g(x):
    def h():
        x = "abc"

    x = x + 1
    print("g: x =", x)
    h()
    return x

x = 3
z = g(x)
```

h код



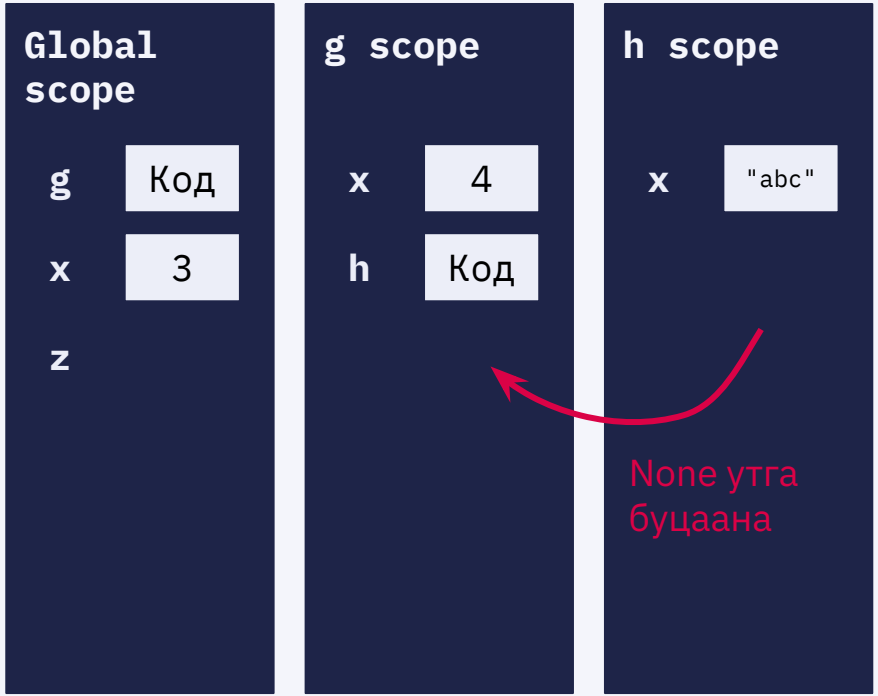
Үйлчлэх хүрээ - Жишээ

```
def g(x):
    def h():
        x = "abc"

    x = x + 1
    print("g: x =", x)
    h()
    return x

x = 3
z = g(x)
```



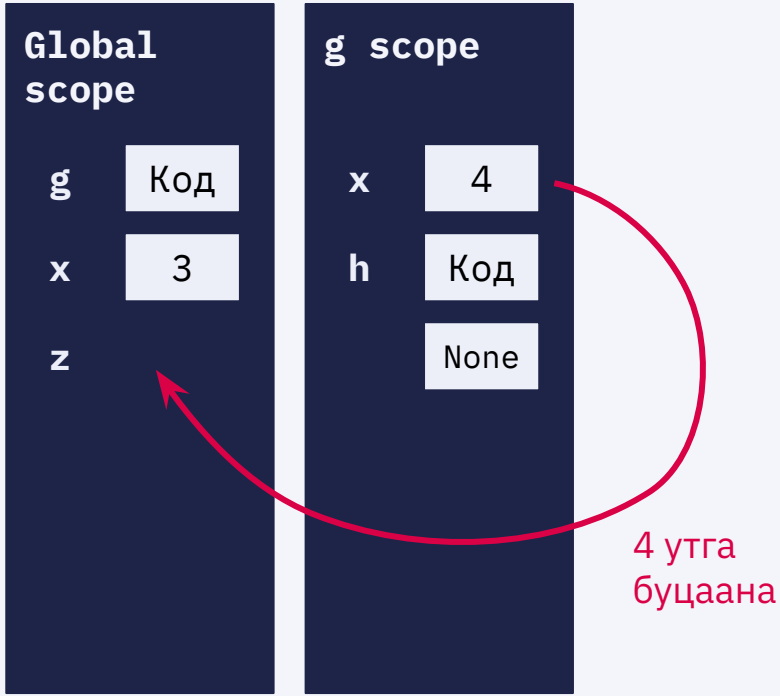
Үйлчлэх хүрээ - Жишээ

```
def g(x):
    def h():
        x = "abc"

    x = x + 1
    print("g: x =", x)
    h()
    return x

x = 3
z = g(x)
```



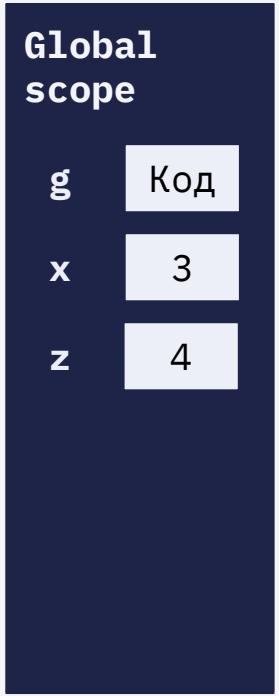
Үйлчлэх хүрээ - Жишээ

```
def g(x):
    def h():
        x = "abc"

    x = x + 1
    print("g: x =", x)
    h()
    return x

x = 3
z = g(x)
```



Үйлчлэх хүрээ - Жишээ

```
def g(x):
    def h():
        x = "abc"

    x = x + 1
    print("g: x =", x)
    h()
    return x

x = 3
z = g(x)
```



Задлах болон хийсвэрлэх

- Кодыг илүү бүтэцлэгдсэн, ойлгоход хялбар болгоно
- Нэг газар л debug хийхэд хангалттай
- Нэг бичээд хэдэн ч удаа, хаана ч ашиглах боломжтой